

Day 04 – SystemVerilog Arrays & Memories

Syllabus: Packed/unpacked arrays; multidimensional arrays; indexing, slicing, part-selects, +:/- :: memories.

Goal: Learn how SystemVerilog stores groups of bits, collections of elements, multidimensional data, and memory structures.

1. What is an Array?

An **array** is a collection of multiple values stored under one variable name. Each element is accessed using an index.

```
int data [0:3];

data[0] = 10;
data[1] = 20;
data[2] = 30;
data[3] = 40;
```

2. Packed Arrays

A **packed array** has dimensions **before** the variable name. Packed dimensions represent bits grouped together as one vector-like object.

```
logic [7:0] data;
data = 8'b1010_1100;

$display("%b", data);
$display("%b", data[7]);
$display("%b", data[0]);
```

Packed arrays are common for buses, registers, instructions, and bit fields.

3. Unpacked Arrays

An **unpacked array** has dimensions **after** the variable name. It represents a collection of separate elements.

```
logic data [0:7];

data[0] = 1'b1;
data[1] = 1'b0;
data[2] = 1'b1;
```

Here there are 8 separate logic elements, each one bit wide.

4. Packed vs Unpacked

Declaration	Meaning
logic [7:0] data;	One 8-bit packed vector
logic data [0:7];	8 separate 1-bit unpacked elements
logic [7:0] memory [0:7];	8 unpacked elements, each 8-bit packed

Golden rule: dimension before the variable name = **packed**; dimension after the variable name = **unpacked**.

5. Packed + Unpacked Together

```
logic [7:0] memory [0:3];

memory[0] = 8'hAA;
memory[1] = 8'hBB;
memory[2] = 8'hCC;
memory[3] = 8'hDD;
```

This means **4 elements × 8 bits**. [7:0] is packed and [0:3] is unpacked.

6. Two Packed Dimensions vs Packed + Unpacked

```
logic [1:0][7:0] data;
logic [7:0] data_array [1:0];

data = 16'hAABB;
// data[1] = AA, data[0] = BB

data_array[1] = 8'hAA;
data_array[0] = 8'hBB;
```

logic [1:0][7:0] data: both dimensions are packed, so it is one 16-bit packed object. **logic [7:0] data_array [1:0]:** one packed dimension plus one unpacked dimension, so it is an unpacked array of two 8-bit elements.

Both contain 16 bits of storage, but their types and indexing/assignment behavior are different.

Day 04 – Indexing, Slicing & Part-Selects

7. Indexing

Indexing selects one element or one bit using an index.

```
logic [7:0] data;
data = 8'b1011_0101;

data[3] // one bit

logic [7:0] memory [0:3];
memory[2] // one 8-bit element
memory[2][5] // bit 5 of element 2
```

8. Slicing

A **slice** selects a range of bits from a packed vector. A common form is `vector[msb:lsb]`.

```
logic [15:0] data;
data = 16'hABCD;

data[15:8] // AB
data[7:0] // CD
```

Slicing is useful for extracting fields from buses, packet headers, addresses, instructions, and data words.

9. Part-Selects

A **part-select** extracts a specified group of bits from a packed vector. A fixed part-select explicitly specifies the range.

```
logic [31:0] data;

data[31:24]
data[23:16]
data[15:8]
data[7:0]
```

If `data = 32'hAABBCCDD`, then `data[31:24]` is AA and `data[7:0]` is DD.

10. Indexed Part-Select: +:

Syntax: `vector[start +: width]`. Start at `start` and select `width` bits toward increasing index values.

```
data[4 +: 4]
```

Selected indices are 4, 5, 6, 7. Therefore `data[4 +: 4]` is equivalent to `data[7:4]`.

```
int start;
start = 8;
data[start +: 8] // data[15:8]
```

11. Indexed Part-Select: -:

Syntax: `vector[start -: width]`. Start at `start` and select `width` bits toward decreasing index values.

```
data[7 -: 4]
```

Selected indices are 7, 6, 5, 4. Therefore `data[7 -: 4]` is equivalent to `data[7:4]`.

12. Why +: / -: are Useful

They are useful when the starting position is variable but the field width is fixed. This is common in packet and bus field extraction.

```
logic [31:0] data;
int start;

data = 32'hAABBCCDD;
start = 0;

$display("%h", data[start +: 8]); // DD

start = 8;
$display("%h", data[start +: 8]); // CC
```

Day 04 – Multidimensional Arrays & Memories

13. Multidimensional Arrays

A **multidimensional array** has two or more dimensions. A 2D array can be visualized like a matrix.

```
int matrix [0:2][0:3];

matrix[0][0] = 10;
matrix[1][2] = 70;

$display("%0d", matrix[1][2]); // 70
```

This creates $3 \times 4 = 12$ integer elements. Access is `matrix[row][column]`.

14. Multidimensional Packed Arrays

```
logic [1:0][7:0] data;
data = 16'hAABB;

$display("%h", data); // AABB
$display("%h", data[1]); // AA
$display("%h", data[0]); // BB
```

Both dimensions are packed. Total width = $2 \times 8 = 16$ bits. This is one packed object, not an unpacked memory.

15. Mixed Multidimensional Arrays

```
logic [7:0] memory [0:3][0:1];

memory[2][1] = 8'h5A;
```

There are 4×2 unpacked elements, and each element is an 8-bit packed value. Total storage = $4 \times 2 \times 8 = 64$ bits.

16. Memories

A **memory** is commonly modeled as an **unpacked array of packed vectors**. The packed dimension represents data width; the unpacked dimension represents the number of locations.

```
logic [7:0] memory [0:255];
```

Meaning: **256 locations $\times$ 8 bits = 2048 bits.**

```
memory[0] // location 0
memory[1] // location 1
memory[255] // location 255
```

17. Memory Read and Write

```
module memory_example;
logic [7:0] memory [0:3];

initial begin
memory[0] = 8'hAA;
memory[1] = 8'hBB;
memory[2] = 8'hCC;
memory[3] = 8'hDD;

$display("memory[0] = %h", memory[0]);
$display("memory[1] = %h", memory[1]);
$display("memory[2] = %h", memory[2]);
$display("memory[3] = %h", memory[3]);
end
endmodule
```

18. Iterating Through a Memory

```
for (int i = 0; i < 4; i++) begin
$display("memory[%0d] = %h", i, memory[i]);
end
```

19. Memory in Verification

A testbench can store expected results in an array. This is a basic idea behind scoreboards and reference models.

```
logic [7:0] expected [0:3];

expected[0] = 8'h10;
expected[1] = 8'h20;
expected[2] = 8'h30;
expected[3] = 8'h40;

if (dut_output == expected[2])
$display("PASS");
else
$display("FAIL");
```

Day 04 – Quick Revision

Quick Revision Table

Concept	Key idea
Array	Collection of values under one variable name
Packed	Dimension before name: logic [7:0] data
Unpacked	Dimension after name: logic data [0:7]
Packed + unpacked	logic [7:0] memory [0:255]
Indexing	Select one bit/element: data[3], memory[2]
Nested indexing	memory[2][5] = bit 5 of element 2
Slicing	Select a range: data[15:8]
Part-select	Select a specified group of packed bits
+:	data[start +: width] → increasing indices
-:	data[start -: width] → decreasing indices
2D array	int matrix [0:2][0:3]
2D access	matrix[row][column]
Memory	Array of packed data elements; e.g. 256 × 8

Key Differences to Remember

logic [7:0] data;
→ one 8-bit packed vector

logic data [0:7];
→ eight separate 1-bit unpacked elements

logic [7:0] memory [0:7];
→ eight unpacked elements, each 8 bits

logic [1:0][7:0] data;
→ two packed dimensions, total 16 bits

logic [7:0] data_array [1:0];
→ two unpacked elements, each 8 bits

Core Rules

- **Before variable name** → packed.
- **After variable name** → unpacked.
- **Indexing** → one element or bit.
- **Slicing / fixed part-select** → group of packed bits.
- **+:** → move toward higher indices.
- **-:** → move toward lower indices.
- **Memory** → commonly an unpacked array of packed data.

Day 4 Practical Focus

Practice these declarations and operations: packed/unpacked arrays, indexing, fixed slices, + :- : part-selects, 2D arrays, memory read/write, and loops through memory.

```
logic [15:0] data;  
logic [7:0] memory [0:3];  
int matrix [0:2][0:3];
```

```
data[15:8];  
data[start +: 8];  
data[start -: 8];
```

```
memory[2];  
memory[2][5];  
matrix[1][2];
```

Final Takeaway

Packed arrays group bits into a vector-like object. Unpacked arrays group elements. Multidimensional arrays add multiple dimensions. Indexing selects elements/bits, slicing and part-selects select groups of packed bits, and memories are commonly modeled as unpacked arrays of packed vectors.